



FPT POLYTECHNIC



LẬP TRÌNH C# 2

BÀI 1: LẬP TRÌNH GENERIC

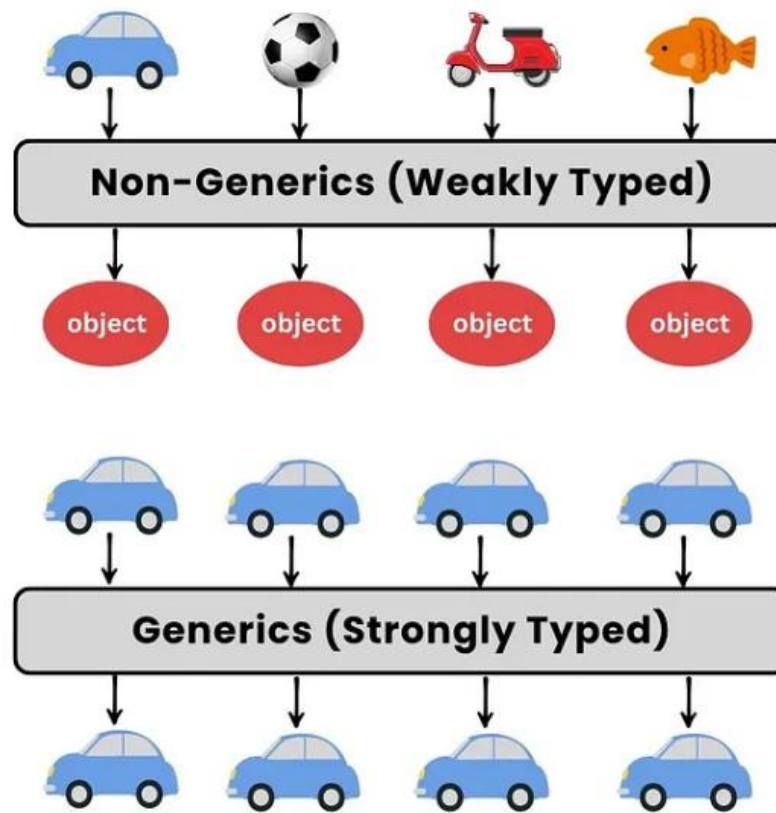
www.poly.edu.vn

- ⦿ Generic interface
- ⦿ Generic class
- ⦿ Generic Constraints
- ⦿ Static class, Partial class



- ❑ Generics và Non-Generics đại diện cho hai cách tiếp cận khác nhau để xử lý các kiểu dữ liệu trong các tập hợp và phương thức

Generics and Non Generics



❑ Non-Generics

- ❖ Không an toàn về kiểu dữ liệu: Sử dụng đối tượng, vì vậy bất kỳ kiểu dữ liệu nào cũng có thể được thêm vào.
- ❖ Cần chuyển đổi kiểu khi sử dụng
- ❖ Rủi ro cao: Dễ xảy ra lỗi trong quá trình thực thi.
- ❖ Vấn đề Boxing/Unboxing: Khi chuyển một kiểu giá trị (như int, double) sang object, .NET phải thực hiện "Boxing" (đóng gói vào vùng nhớ Heap). Việc này rất tốn tài nguyên

```
ArrayList list = new ArrayList();  
list.Add(10); // Can add any type  
int number = (int)list[0]; // Manual casting needed
```

❑ Generics

- ❖ An toàn kiểu dữ liệu: tự định nghĩa kiểu dữ liệu khi sử dụng
- ❖ Không cần ép kiểu: Ngăn ngừa lỗi thời gian chạy do ép kiểu không hợp lệ
- ❖ Có thể tái sử dụng: Viết mã hoạt động với bất kỳ kiểu dữ liệu nào
- ❖ Kiểm tra lỗi trong quá trình biên dịch: Lỗi được phát hiện sớm.

- ❑ Là lớp được thiết kế để hoạt động với nhiều kiểu dữ liệu khác nhau mà không cần viết lại mã nguồn
- ❑ Thay vì chỉ định một kiểu dữ liệu cụ thể, sử dụng một hoặc nhiều **tham số kiểu (type parameters)** làm placeholder cho các kiểu dữ liệu đó.

```
public class TênLớpGeneric<T>
{
    // Các thành viên (fields, properties, methods) sử dụng T
}
```

Demo xây dựng một generics ResponseModel<T>

```
// Lớp mô phỏng dữ liệu người dùng
public class User
{
    public string FullName { get; set; } = string.Empty;
    public override string ToString() => FullName;
}

using System.Collections.Generic;
namespace DotNetGenerics
{
    // Lớp Generic
    public class ResponseModel<T>
    {
        // Thuộc tính Data có kiểu là T (linh hoạt tùy theo đối tượng trả về)
        public T Data { get; set; }
        public bool IsSuccess { get; set; }
        public string Message { get; set; }
        public DateTime Timestamp { get; set; }
        public ResponseModel(T data, bool isSuccess, string message)
        {
            Data = data;
            IsSuccess = isSuccess;
            Message = message;
            Timestamp = DateTime.Now;
        }
        public void PrintResponse()
        {
            Console.WriteLine($"[{Timestamp:HH:mm:ss}] Success: {IsSuccess}");
            Console.WriteLine($"Message: {Message}");
            Console.WriteLine($"Data Type: {typeof(T).Name}");
            Console.WriteLine($"Data Value: {Data}");
            Console.WriteLine("-----");
        }
    }
}
```

❑ Demo xây dựng một generics ResponseModel<T>

```

Console.WriteLine("Generic Class Demo (Explicit Typing) ===\n");
// 1. Sử dụng với kiểu dữ liệu cơ bản (string)
ResponseModel<string> stringResponse = new ResponseModel<string>(
    "Token_ABC_123",
    true,
    "Login Successful"
);
stringResponse.PrintResponse();
// 2. Sử dụng với kiểu số nguyên (int)
ResponseModel<int> intResponse = new ResponseModel<int>(
    200,
    true,
    "Status Code"
);
intResponse.PrintResponse();
// 3. Sử dụng với đối tượng phức tạp (User custom class)
User newUser = new User { FullName = "Thepv" };
ResponseModel<User> userResponse = new ResponseModel<User>(
    newUser,
    true,
    "User data retrieved"
);
userResponse.PrintResponse();
// 4. Sử dụng với một Danh sách (List<T>) - Generic lồng nhau
List<string> tags = new List<string> { "DotNet9", "C#13", "Generics" };
ResponseModel<List<string>> listResponse = new ResponseModel<List<string>>(
    tags,
    true,
    "Tags fetched"
);
List<string> resultData = listResponse.Data;
Console.WriteLine($"Total tags: {resultData.Count}");
  
```


- ❑ Là một interface chứa các tham số kiểu ($<T>$) định nghĩa một "bản thiết kế" cho các hành động mà không quan tâm đối tượng dữ liệu cụ thể là gì.
- ❑ Tính trừu tượng cao: định nghĩa các hành vi chung (như Thêm, Sửa, Xóa) cho mọi loại dữ liệu
- ❑ Dependency Injection (DI): triển khai DI trong ASP.NET Core, đăng ký $IRepository<T>$ và dùng chung cho nhiều Service khác nhau.
- ❑ Đảm bảo tính nhất quán: Ép buộc các lớp thực thi phải tuân thủ cùng một cấu trúc logic

❑ Demo tạo Interface cho các thao tác lưu trữ

```
public interface IRepository<T>
{
    void Add(T item);
    T GetById(int id);
    IEnumerable<T> GetAll();
}
```

❑ Thực thi Interface

```
public class GenericRepository<T> : IRepository<T>
{
    private readonly List<T> _data = new List<T>();

    public void Add(T item)
    {
        _data.Add(item);
    }

    public T GetById(int id)
    {
        if (id >= 0 && id < _data.Count)
        {
            return _data[id];
        }
        return default(T);
    }

    public IEnumerable<T> GetAll()
    {
        return _data;
    }
}
```

- ❑ Constraints là cơ chế cho phép giới hạn các kiểu dữ liệu mà tham số $\langle T \rangle$ có thể tiếp nhận
- ❑ Thay vì để T là "bất cứ gì", có thể ép buộc T phải là một Class, một Interface cụ thể, hoặc phải có một hàm khởi tạo
- ❑ An toàn từ lúc biên dịch (Compile-time Safety): Ngăn chặn việc truyền sai kiểu dữ liệu ngay khi viết code
- ❑ Truy cập thành viên: Nếu ràng buộc where T : $\text{Entity} \Rightarrow$ có thể gọi $T.Id$ trực tiếp trong code

□ Tổng hợp một số ràng buộc quan trọng

Từ khóa	Ý nghĩa
<code>where T : struct</code>	T phải là kiểu giá trị (Value Type) như <code>int</code> , <code>double</code> , <code>enum</code> .
<code>where T : class</code>	T phải là kiểu tham chiếu (Reference Type).
<code>where T : notnull</code>	T không được là kiểu nullable (Bắt đầu có từ C# 8.0/ .NET Core 3.0).
<code>where T : new()</code>	T phải có hàm khởi tạo công khai không tham số.
<code>where T : <BaseClassName></code>	T phải là hoặc kế thừa từ lớp cha được chỉ định.
<code>where T : <InterfaceName></code>	T phải thực thi interface được chỉ định.
<code>where T : unmanaged</code>	T là kiểu dữ liệu không được quản lý (dành cho tối ưu hiệu năng thấp/interop).

- ❑ Demo xây dựng hệ thống quản lý dữ liệu dùng lớp DataService chỉ làm việc với các đối tượng có ID và có thể khởi tạo được.

```
using System.Collections.Generic;
public interface IEntity
{
    int Id { get; set; }
    string GetSummary();
}
public class Product : IEntity
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public Product()
    {
    }
    public string GetSummary()
    {
        return "Product #" + Id + ": " + Name + " - $" + Price;
    }
}
```

// 3. Sử dụng ràng buộc

```
public class DataService<T> where T : class, IEntity, new()
```

```
{
```

```
    private readonly List<T> _items = new List<T>();
```

```
    public void AddNewItem(int id, string info)
```

```
    {
```

```
        T newItem = new T();
```

```
        newItem.Id = id;
```

```
        _items.Add(newItem);
```

```
        Console.WriteLine("[System] Đã thêm mới: " + newItem.GetSummary());
```

```
    }
```

```
    public T GetById(int id)
```

```
    {
```

```
        foreach (T item in _items)
```

```
        {
```

```
            if (item.Id == id)
```

```
            {
```

```
                return item;
```

```
            }
```

```
        }
```

```
        return null;
```

```
    }
```



```
DataService<Product> productService = new DataService<Product>();
```

```
productService.AddNewItem(101, "Laptop Dell");
```

```
productService.AddNewItem(102, "Mechanical Keyboard");
```

```
Product foundProduct = productService.GetById(101);
```

```
if (foundProduct != null)
```

```
{
```

```
    Console.WriteLine("Tìm thấy: " + foundProduct.Name);
```

```
}
```



DEMO

- Hiện thực các ví dụ





FPT POLYTECHNIC



LẬP TRÌNH C# 2

BÀI 1: LẬP TRÌNH GENERIC(P2)

www.poly.edu.vn

- ❑ Nhắc lại các thuộc tính, phương thức:
 - ❖ Chỉ có thể sử dụng sau khi khởi tạo đối tượng.
 - ❖ Dữ liệu thuộc về riêng mỗi đối tượng (xét cùng 1 thuộc tính thì các đối tượng khác nhau thì thuộc tính đó sẽ mang các giá trị khác nhau)
 - ❖ Được gọi thông qua tên của đối tượng.
- ❑ Mong muốn **1 thuộc tính nào đó được dùng chung cho mọi đối tượng** (chỉ được cấp phát 1 vùng nhớ duy nhất) → **thành viên tĩnh**

❑ Đặc điểm của thành viên tĩnh:

- ❖ Được khởi tạo 1 lần duy nhất ngay khi biên dịch chương trình.
- ❖ Có thể dùng chung cho mọi đối tượng.
- ❖ Được gọi thông qua tên lớp.
- ❖ Được huỷ khi kết thúc chương trình.

❑ Có 4 loại thành viên tĩnh chính:

- ❖ Biến tĩnh (static variable).
- ❖ Phương thức tĩnh (static method).
- ❖ Phương thức khởi tạo tĩnh (static constructor).
- ❖ Lớp tĩnh (static class).

❑ Biến tĩnh: - Cú pháp khai báo:

```
<phạm vi truy cập> static <kiểu dữ liệu> <tên biến> = <giá trị khởi tạo>;
```

❑ Một số đặc điểm của **biến tĩnh**:

- ❖ **Biến static** có thể được khởi tạo bên ngoài hàm thành viên, hoặc ngoài định nghĩa class, hoặc trong định nghĩa class.
- ❖ Là biến **dùng chung cho mọi đối tượng thuộc lớp**.
- ❖ Được khởi tạo vùng nhớ một lần duy nhất ngay khi chương trình được nạp vào bộ nhớ để thực thi. Và nó có vùng nhớ riêng, không bị thay đổi.
- ❖ Được **gọi trực tiếp thông qua tên lớp mà không cần tạo đối tượng của lớp**, chính vì điều này mà biến static thường được sử dụng để định nghĩa cho các hằng số (constant)..

❑ Biến tĩnh:

```
namespace C#2Demo;
public class People(int id, string name)
{
    private static string school = "FPoly";

    public void Show()
    {
        Console.WriteLine($"Id: {id},\nName: {name},\nschool: {school}\n");
    }
}

public class Student
{
    public static void PrintName(string firstName, string lastName)
    {
        Console.WriteLine($"Student name: {firstName} {lastName}");
    }
}

People p1 = new(1, "PhanVietThe");
p1.Show();

People p2 = new(2, "Nguyen Van B");
p2.Show();

People p3 = new(3, "Nguyen Van C");
p3.Show();

Student.PrintName("Fpoly", "Cntt");
Student.PrintName("Welcome to", "Fpoly");
```

```
Id: 1,
Name: Nguyen Van A,
school: FPoly
```

```
Id: 2,
Name: Nguyen Van B,
school: FPoly
```

```
Id: 3,
Name: Nguyen Van C,
school: FPoly
```

❑ Phương thức tĩnh: - Cú pháp khai báo:

```
<phạm vi truy cập> static <kiểu dữ liệu trả về> <tên phương thức>;  
{  
    // nội dung phương thức  
}
```

❑ Một số đặc điểm của **phương thức tĩnh**:

- ❖ **Static method** là một phương thức dùng chung của lớp. **Được gọi thông qua tên lớp và không cần khởi tạo bất kỳ đối tượng nào**, từ đó tránh việc lãng phí bộ nhớ..
- ❖ Hỗ trợ trong việc viết các hàm tiện ích của thư viện để sử dụng lại.
- ❖ Trong phương thức có sử dụng biến static thì phương thức đó cũng phải được khai báo là static.

❑ Phương thức tĩnh:

```
using System;

namespace DemoC#2;

public class Student
{
    public static void PrintName(string firstName, string lastName)
    {
        string name = $"{firstName} {lastName}";
        Console.WriteLine($"Student name: {name}");
    }
}

Student.PrintName("Fpoly", "Cntt");
Student.PrintName("Welcome to", "Fpoly");
```

❑ Phương thức khởi tạo tĩnh:

- Cú pháp khai báo:

```
static <tên lớp>;  
{  
    // nội dung của hàm dựng, hàm khởi tạo constructor  
}
```

❑ Một số đặc điểm của **phương thức khởi tạo tĩnh**:

- ❖ Không được phép khai báo phạm vi truy cập
- ❖ Static constructor sẽ được gọi một lần duy nhất khi chương trình được nạp lên.
- ❖ Trong phương thức có sử dụng biến static thì phương thức đó cũng phải được khai báo là static.

❑ Phương thức khởi tạo tĩnh:

```
public class G1(int j)
{
    static G1()
    {
        Console.WriteLine("Example of Static Constructor");
    }

    public string G1Detail(string name, string branch)
    { $"Name: {name} Branch: {branch}"; }

    public static void RunDemo()
    {
        G1 obj = new(1);
        Console.WriteLine(obj.G1Detail("Sunil", "CSE"));

        G1 ob = new(2);
        Console.WriteLine(ob.G1Detail("Sweta", "ECE"));
    }
}
```


❑ Lớp tĩnh:

- ❖ static class chỉ chứa thành viên là static
- ❖ static class không có thể hiện.
- ❖ static class không chứa hàm xây dựng.
- ❖ static class thường được dùng với mục đích khai báo một lớp tiện ích chứa các hàm tiện ích hoặc hằng số.

❑ Lớp tĩnh:

```
public static class MyMath
{
    public static float PI = 3.14f;

    public static int Squared(int n)
    {
        return n * n;
    }
}

int squared;
float pi = MyMath.PI;
squared = MyMath.squared(10);
Console.WriteLine("squared: " + squared);
Console.WriteLine("\nPI: " + pi);
Console.ReadKey();
```

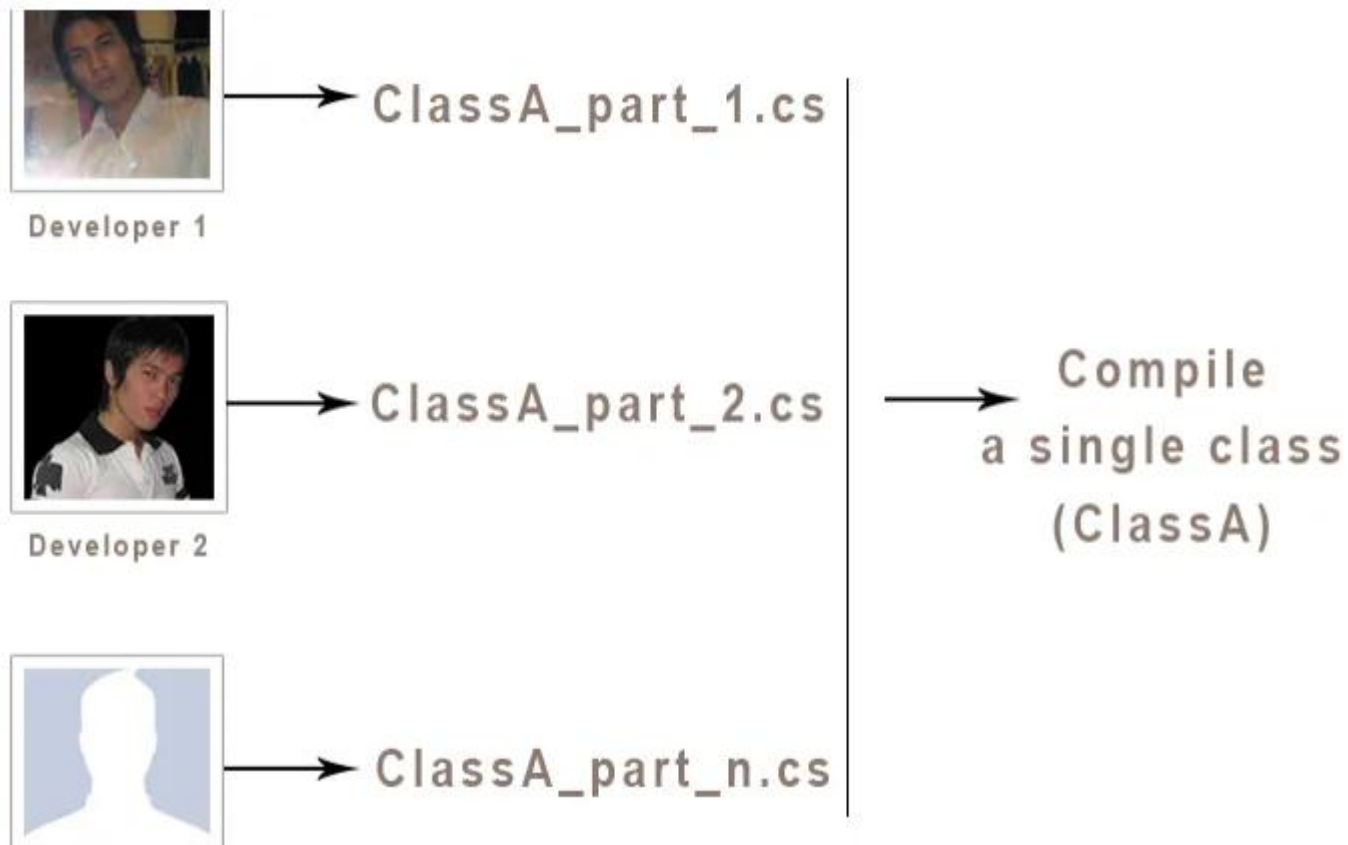


DEMO

- Hiện thực các ví dụ



- ❑ Partial Class trong C# là một tính năng giúp chúng ta chia một class thành hai hay nhiều phần hay file khác nhau.



```
public partial class MyPartialClass
{
    partial void PartialMethod(int val);

    public MyPartialClass()
    {
    }

    public void Method2(int val)
    {
        Console.WriteLine(val);
    }
}
```

PartialClassFile1.cs

```
public partial class MyPartialClass
{
    public void Method1(int val)
    {
        Console.WriteLine(val);
    }

    partial void PartialMethod(int val)
    {
        Console.WriteLine(val);
    }
}
```

PartialClassFile2.cs

Compiles as a single class

```
public class MyPartialClass
{
    public MyPartialClass()
    {
    }

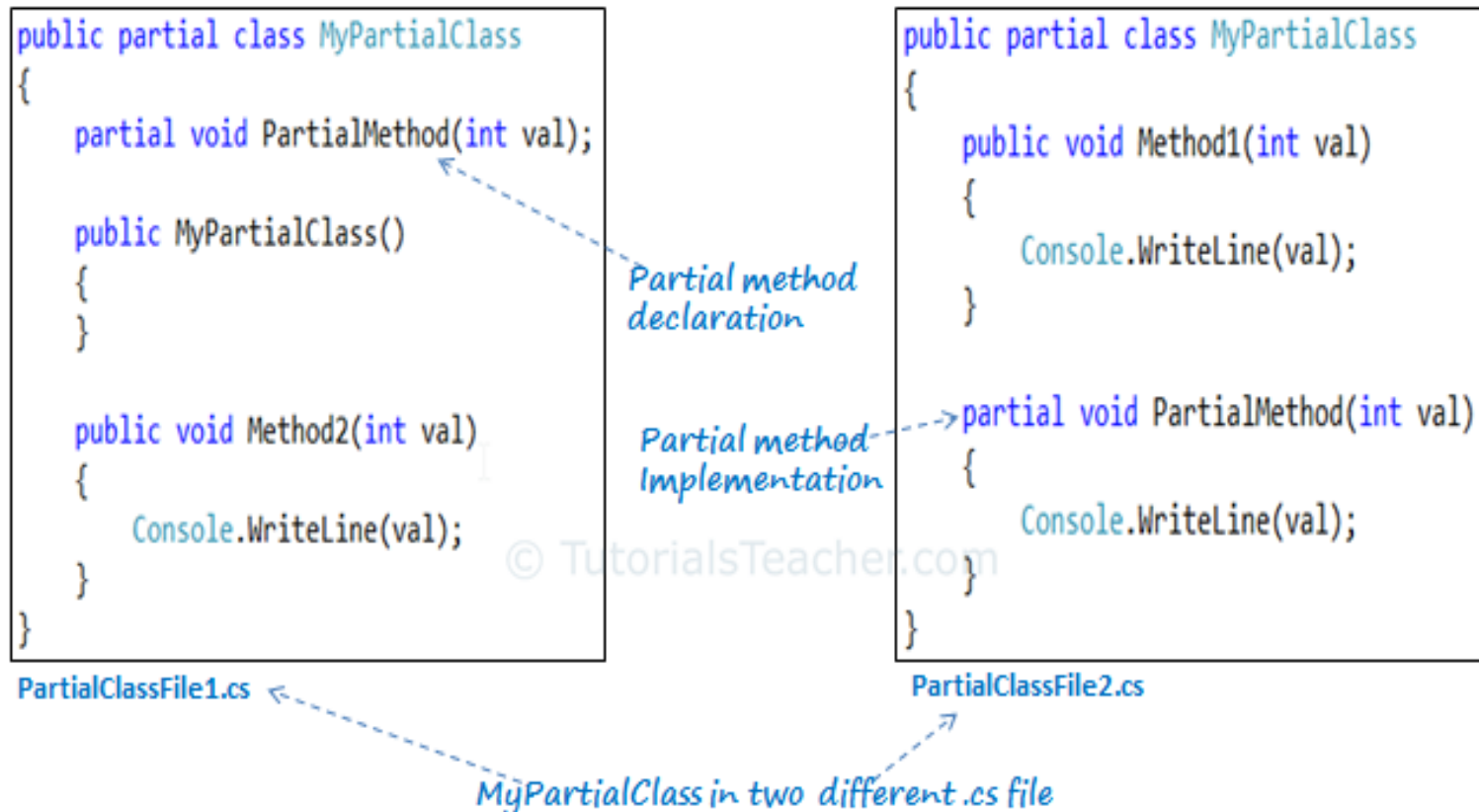
    public void Method1(int val)
    {
        Console.WriteLine(val);
    }

    public void Method2(int val)
    {
        Console.WriteLine(val);
    }

    private void PartialMethod(int val)
    {
        Console.WriteLine(val);
    }
}
```

Partial Class

- Partial method cho phép code sinh tự động gọi phương thức nhưng không nhất thiết phải xây dựng (implement) phương thức đó.





DEMO

- Hiện thực các ví dụ



Tổng kết bài học

- Generic interface
- Generic class
- Generic Constraints
- Static class, Partial class





KẾT THÚC